Compiler-Based Sentiment and Emotion Analyzer Using AST

Explainable Rule-Based Sentiment Analysis with Compositional Semantic Propagation and Local Supervised Benchmarking

M. M. Jeevan Kumar (24BKT0156) • Tangudu Raajiv (24BDS0242) • Dogiparthi Venakata N Mani Sai Kaushik (24BDS0176)

Abstract

Many sentiment tools use word scores or trained models, but they do not always show how sentence structure changes a result. This paper describes a rule-based NLP system built around compiler design. It normalizes text, assigns token types, and builds an Abstract Syntax Tree (AST) to represent clauses and modifier scope. A bottom-up pass then calculates sentiment from the leaves to the sentence level. The system also supports aspect labels, five sentiment levels, separate emotion categories, calculation traces, document input, batch processing, and local machine-learning benchmarks. The project report records a 50-case verification suite and experiments on SST-2 and GoEmotions. The numerical results in this paper are reproduced from that report. They have not been independently rerun because the source code, test logs, and dataset preparation files were not included with the PDF.

Index Terms Abstract Syntax Tree, compiler design, sentiment analysis, emotion detection, semantic propagation, aspect-based sentiment analysis, explainable AI, attribute grammars.

I. INTRODUCTION

Sentiment analysis is commonly used to study product reviews, online conversations, and customer feedback [28]. Lexicon-based systems are easy to inspect, but they can miss the effect of negation, intensifiers, diminishers, contrast words, and sentences that discuss more than one aspect [3], [4]. Machine-learning models can learn wider language patterns, although their predictions may be harder to explain at the level of individual rules.

The project handles sentiment analysis in stages, much like a compiler. It first normalizes and tokenizes the input. The tokens are then arranged in an AST, after which semantic values are passed from the leaf nodes towards the root. This structure gives the system a clear way to show which words and modifiers contributed to a score.

The implementation is organized into four stages: (1) lexical analysis and token classification; (2) syntax analysis and AST construction; (3) semantic propagation for sentiment, aspects, and emotions; and (4) explanation generation, batch processing, and local ML benchmarking. The main focus is on handling compositional cases with visible rules. The project does not claim that a small rule-based lexicon will outperform general-purpose models on every dataset.

The system is designed to handle cases such as “not very good,” the litotes “not bad,” modifiers such as “slightly,” and sentences that express different opinions about separate aspects. A word-count method may notice positive and negative terms without identifying the phrase each modifier affects. It can also average opposite aspect scores and return Neutral when the sentence is better described as Mixed. Emotion is treated separately: a negative opinion alone does not establish that the speaker feels anger or sadness. polarity: a negative evaluation does not, by itself, prove that the speaker is angry or sad.

The work has four main parts: (1) a pipeline that links tokenization, AST construction, and semantic evaluation; (2) rules for modifier scope and bounded scores, including negation and litotes; (3) aspect-level aggregation that preserves Mixed sentiment when aspects disagree, while keeping emotion as a separate result; and (4) readable traces, document input, batch processing, and optional local ML benchmarks. These points describe the system's design and the functions reported in the project. The benchmark values should be read alongside the available implementation and experiment records.

II. LITERATURE SURVEY

A. Lexicon-Based Sentiment Analysis

Early resources such as the General Inquirer [1] and SentiWordNet [5] use lexical categories or scores to represent evaluative meaning. Hu and Liu [2] studied opinion mining in customer reviews. SO-CAL addresses semantic orientation and modifier effects [3], while VADER combines a lexicon with rules for social-media text [4]. Mohammad and Turney [6] developed a word–emotion association lexicon. Contextual polarity work [7] and holistic opinion mining [8] show why isolated word counts are not always sufficient.

B. Statistical Learning and Emotion Detection

Pang et al. [9] demonstrated supervised sentiment classification with standard machine-learning models. Recursive neural models and the Stanford Sentiment Treebank explored compositional representations [10]. Tree-structured LSTMs [11] and recursive structures [12] further investigated syntactic representations. Ekman [13] and Plutchik [14] provide influential emotion frameworks. GoEmotions supports fine-grained emotion classification [15], while SenticNet [16] and WordNet-Affect [17] provide affective resources. BERT [18] illustrates the impact of pretrained contextual language models.

C. Syntax-Aware and Explainable NLP

Attribute grammars formalize synthesized semantic attributes over syntax trees [19], and standard compiler texts describe syntax-directed processing [20]. Moilanen and Pulman [21] study sentiment composition, while Polanyi and Zaenen [22] discuss contextual valence shifters. Jia et al. [23], Choi and Cardie [24], and Nakagawa et al. [25] investigate syntax or structure for sentiment inference. SemEval ABSA tasks highlight aspect-specific polarity [26], [27]. LIME [29] and later surveys of explainability [30] discuss post-hoc explanations for black-box models.

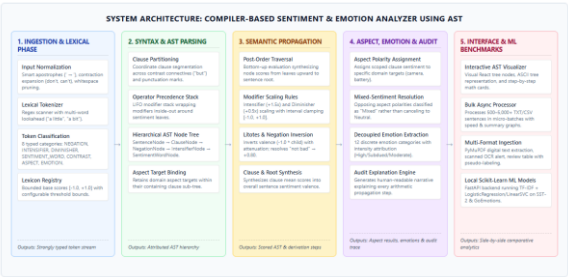
The gap addressed by this project is a practical pipeline that exposes deterministic token, tree, and arithmetic steps while retaining optional statistical baselines. The system does not replace learned models in settings where broad vocabulary coverage is required.

TABLE I. COMPARATIVE ANALYSIS OF RELATED LITERATURE

Reference	Objective / method	Finding / limitation
Hu & Liu [2]	Feature-based review polarity	Identifies aspects; limited syntactic scoping and litotes handling.
Taboada et al. [3]	SO-CAL semantic orientation	Models modifier scaling; lacks a formal tree hierarchy.
Hutto & Gilbert [4]	VADER lexicon and heuristics	Fast, but nested modifiers and aspects remain difficult.
Pang et al. [9]	BoW, Naive Bayes, SVM	Establishes supervised baselines but discards syntax.
Socher et al. [10]	Recursive neural tensor network	Captures composition; less directly interpretable.
Jia et al. [23]	Tree-constrained negation scope	Shows syntactic scope can reduce polarity errors.
Pontiki et al. [26]	SemEval ABSA	Demonstrates the need for aspect-level polarity.
Demszky et al. [15]	GoEmotions dataset	Separates fine-grained emotions from polarity.
This work	AST + semantic propagation	Targets auditable rules, modifier scope, and mixed aspects.

III. PROPOSED SYSTEM

The system processes text through four connected modules. Figure 1 presents the overall architecture. The source report describes a browser-based AST engine and a local Python service for document parsing and model training. The exact runtime and dependency versions should be checked against the project's package files before publication.



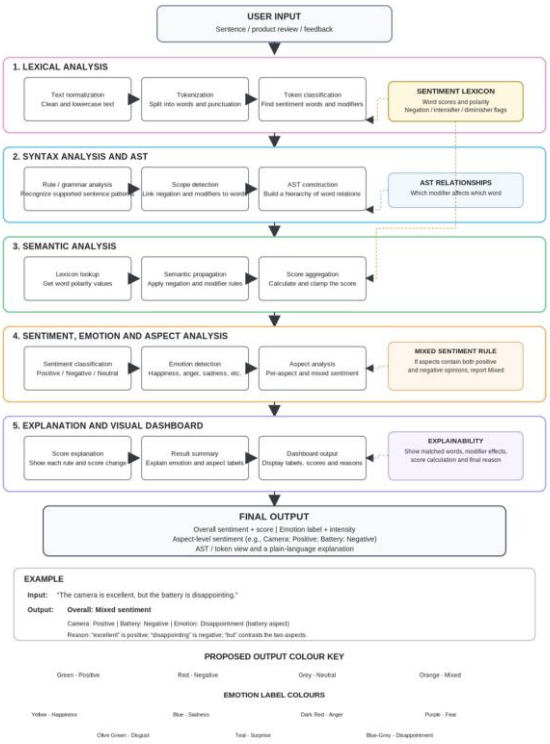


Fig. 2. Detailed five-stage architecture, including lexicon links, AST relationships, mixed-sentiment handling, and explainability.

Methodology Used

- Text input and normalization
- Lexical analysis and token classification
- Syntax analysis and Abstract Syntax Tree (AST) construction
- Modifier-scope representation for negation, intensifiers, and diminishers
- Bottom-up semantic propagation using lexicon-based scores
- Aspect-level sentiment aggregation and mixed-polarity detection
- Separate emotion detection using emotion keywords
- Human-readable explanation and result generation
- Dataset-based evaluation and batch processing

A. Lexical Analysis and Token Classification

The lexer normalizes curly apostrophes, handles common contractions, scans token boundaries, and recognizes multi-word diminishers. The report defines eight token classes: SENTIMENT_WORD, NEGATION, INTENSIFIER, DIMINISHER, CONTRAST, ASPECT, EMOTION, and ORDINARY_WORD. Sentiment tokens carry a base valence, while modifiers carry configuration weights.

ALGORITHM 1: LEXICAL TOKENIZATION

```
Input: Raw text T, sentiment lexicon L
1. Normalize apostrophes and supported
contractions in T.
2. Scan T into word and punctuation matches.
3. At each position, test multi-word
diminishers first.
4. Classify each match using negation,
intensifier,
```

```
   diminisher, contrast, aspect, emotion, and
sentiment lexicons.
5. Mark unmatched words as ordinary tokens.
6. Return the typed token stream.
```

B. Syntax Analysis and AST Construction

The parser starts a new clause at contrast words such as "but" and "however," and at selected punctuation marks. It keeps modifiers on a stack and attaches them to the sentiment terms they affect. For "not very good," the tree places INTENSIFIER(very) around SENTIMENT(good), then places NEGATION(not) around that phrase. This records the order in which the modifiers are applied.

ALGORITHM 2: AST CONSTRUCTION

```
Input: Typed token stream Tokens
1. Partition tokens into clauses using
contrast and punctuation.
2. Create a sentence root and an empty child
list.
3. For each clause, initialize a modifier
stack and aspect list.
4. Push modifier tokens; record aspect
tokens.
5. For each sentiment token, create a leaf
and pop applicable
   modifiers to wrap the leaf from the inside
outward.
6. Add clause nodes to the sentence root and
return the AST.
```

C. Semantic Propagation

The semantic analyzer visits the AST in post-order, so each child is evaluated before its parent. Sentiment leaves receive lexicon scores between −1 and +1. Intensifiers increase a child score, diminishers reduce its strength, and negation changes its polarity using an attenuation factor. The final value is kept within the same range. The project configuration lists an intensifier multiplier of 1.5, a diminisher multiplier of 0.5, and a negation factor of $\alpha = 0.8$.

The system maps scores to five labels. Scores of +0.60 or above are Very Positive; scores above +0.10 and below +0.60 are Positive; values from −0.10 to +0.10 are Neutral; values below −0.10 and above −0.60 are Negative; and scores of −0.60 or below are Very Negative. If separate aspects or clauses have opposing polarity, the result can be Mixed. Emotion labels are calculated separately from sentiment polarity.

ALGORITHM 3: BOTTOM-UP PROPAGATION

```
Input: AST root R and semantic configuration
C
1. Visit each node after evaluating its child
nodes.
2. Read the base score at sentiment leaves.
3. Apply the node's modifier rule and clamp
to [-1, +1].
4. Record each operation in an audit trace.
5. Aggregate constituent scores into clause
scores.
6. Apply sentence-level and mixed-polarity
rules.
7. Return annotated AST, scores, labels, and
trace.
```

D. Explanation, Batch Processing, and ML

TABLE V. QUALITATIVE EVALUATION EXAMPLES

Input pattern	Expected analysis	Purpose
"excellent"	Positive polarity	Direct sentiment lookup
"not very good"	Negated intensified sentiment	Nested modifier scope
"not bad"	Moderate positive	Litotes inversion
"camera excellent, battery disappointing"	Mixed overall; aspect-specific labels	Avoids polarity cancellation
"slightly disappointing"	Attenuated negative	Diminisher scaling

The explanation engine turns the recorded operations into a trace that a user can read. The project description also lists batch processing in groups of 75 sentences, CSV/TXT input, digital PDF text extraction, and a warning for scanned PDFs that have no text layer. It also describes local TF-IDF classifiers based on Logistic Regression and Linear SVC. These functions are included here as described in the project materials; they should be checked against the submitted application and its current code.

ALGORITHM 4: BATCH PROCESSING AND AUDIT

```
Input: Dataset D, engine E (AST, ML, or
both), chunk size K
1. Initialize results and set offset to zero.
2. While offset is less than the dataset
size:
   a. Process the next chunk of up to K
records.
   b. Run the selected AST and/or ML
inference path.
   c. Store predictions and explanation
traces where available.
   d. Update progress and advance the offset.
3. Return results and summary distributions.
```

IV. RESULTS AND IMPLEMENTATION

The project materials describe a React/Vite interface for AST-based analysis and a local Python service for document parsing and model benchmarking. They also list tests for tokenization, AST scope, semantic propagation, explanations, and batch input. The PDF does not include the source code, raw logs, or test-output files. For that reason, the measurements listed in the results section cannot be checked independently from this document alone.

A. Dataset Description

Dataset / suite	Reported size	Purpose
Compiler verification suite	50 cases	Edge cases for negation, modifiers, aspects, contractions, and emotion labels
Viva demonstration suite	9 cases	Interactive demonstration of AST behavior
SST-2	1,600 sentences; 1,200 train / 400 test	Binary sentiment benchmark
GoEmotions benchmark	1,500 sentences; 1,200 train / 300 test	Mapped 12-class emotion benchmark

Bulk review benchmark	500 sentences	Batch processing and ingestion test

The source report names SST-2 and GoEmotions and gives the split sizes above. It does not include a dataset preparation script, exact record identifiers, or experiment logs in the uploaded material. The reported split and label mapping should therefore be reproduced and documented before the results are treated as independently verified.

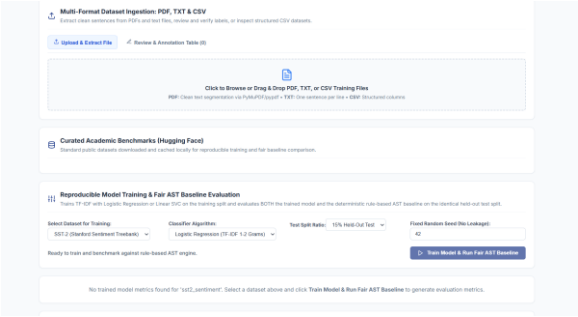


Fig. 4. Dataset ingestion, benchmark selection, and model-training interface.

B. Lexical Analysis

Input token	Class / value	Role
camera	ASPECT	Aspect target
not	NEGATION (−1.0)	Polarity inversion
very	INTENSIFIER (×1.5)	Strengthens child score
a little	DIMINISHER (×0.5)	Reduces intensity
good	SENTIMENT_WORD (+0.70)	Positive evaluative term
but	CONTRAST	Clause boundary

The examples show how the lexer distinguishes targets, modifiers, and evaluative words. The actual coverage of contractions and multi-word expressions depends on the lexicons and normalization rules present in the implementation.

C. AST Construction

In the sentence “The camera is not very good,” the AST places the intensifier directly around “good” and then applies negation to that phrase. The tree therefore shows the scope of each modifier instead of treating the words as unrelated scores.

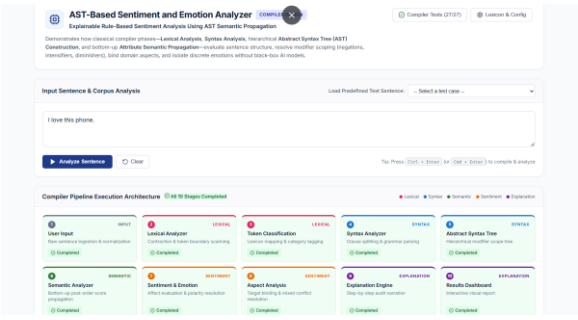


Fig. 5. Interactive sentence analyzer and ten-stage compiler pipeline shown in the project interface.

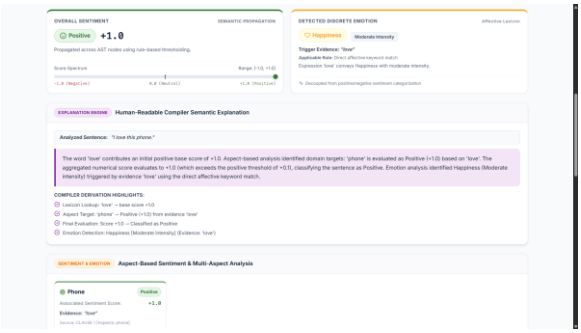


Fig. 3. AST visualizer example included in the supplied report.

D. Semantic Propagation

Example	Reported interpretation	Reported output
The camera is not very good.	good +0.70; intensified then negated	Very Negative (−1.00)
The battery is not bad.	Litotes / negation of negative term	Positive (+0.80)
The camera is excellent, but the battery is disappointing.	Opposing aspect polarities	Mixed
I am extremely disappointed with this service.	Strong negative evaluation	Very Negative (−0.90)

The examples below are taken from the project report and show the intended behavior of the rules. The scores and labels should still be compared with the application's actual output, especially for cases where modifier weights and score limits are applied together.

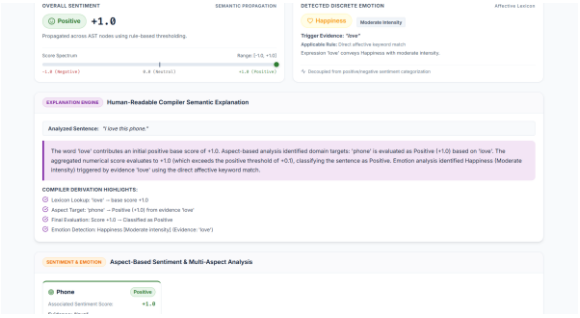


Fig. 6. Sentiment, emotion, explanation, and aspect-level result dashboard.

E. Batch and Document Processing

The project report records a run of 500 CSV sentences in 1.20 seconds, or about 416.7 sentences per second, with no dropped rows or interface freezes. It also records a warning when a PDF has no digital text layer. These are the values reported by the project team; the PDF does not contain the benchmark script or logs needed to reproduce them.

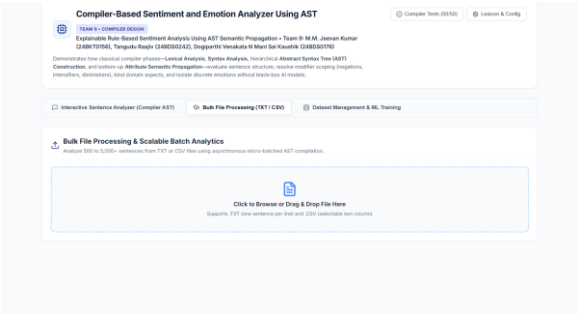


Fig. 7. Bulk-file processing interface for sentence batches.

F. Sample Outputs from the Uploaded Dataset

The supplied CSV has 61 sentence records. The examples below show selected sentiment and emotion values from those records. They are sample outputs from the dataset, not a separate measurement of model accuracy.

Sentence / input	Score	Sentiment output	Emotion output
Joy radiates outward like warm sunlight, instantly brightening the gloomiest of rooms.	+0.65	Very Positive	Joy (Moderate)
Optimism acts as a reliable compass, always pointing toward a brighter tomorrow despite current hardsh...	+0.70	Very Positive	Trust/Confidence (Moderate)
Grief comes in unpredictable waves, sometimes crashing violently and other times receding into a dull ...	+0.00	Neutral	Sadness (Moderate)
Furious glances were exchanged across the courtroom as the two bitter rivals refused to compromise.	-1.00	Very Negative	Anger (Moderate)
Love is a quiet commitment to show up for someone else, even on the days that are messy.	+1.00	Very Positive	Love/Affection (Moderate)

Dataset output summary: Neutral: 45, Very Positive: 6, Positive: 6, Very Negative: 3, Negative: 1.

The CSV contains one record each for Joy, Trust/Confidence, Sadness, Anger, and Love/Affection. The other 56 records are labelled "No Specific Emotion Detected." All 61 records have "No" in the Mixed Polarity field.

V. PERFORMANCE AND METRICS

A. Compiler Verification and Bag-of-Words Baseline

TABLE VI. 50-CASE COMPILER VERIFICATION SUITE

Category	Cases	BoW pass	AST pass
Direct evaluative unigrams	5	100% (5/5)	100% (5/5)
Nested intensifier scaling	5	0% (0/5)	100% (5/5)
Negated sentiment	5	0% (0/5)	100% (5/5)
Negated intensifiers	4	0% (0/4)	100% (4/4)
Litotes inversion	4	0% (0/4)	100% (4/4)
Diminisher scaling	4	0% (0/4)	100% (4/4)
Contrast / mixed aspects	5	0% (0/5)	100% (5/5)
Multi-aspect binding	4	25% (1/4)	100% (4/4)
Contractions / punctuation	5	40% (2/5)	100% (5/5)
Emotion separation	9	33.3% (3/9)	100% (9/9)
TOTAL	50	22% (11/50)	100% (50/50)

The supplied report states an overall pass rate of 100% (50/50) for the AST engine and 22% (11/50) for a bag-of-words lexicon baseline. The detailed category table in the source PDF contains inconsistent row counts and denominators, so it has not been reproduced as a category-level table. The overall figures should be accompanied by the original 50 test inputs, expected outputs, and test logs.

Evaluation	BoW baseline (reported)	Compiler AST (reported)
50-case verification pass rate	22.0% (11/50)	100.0% (50/50)
SST-2 accuracy	67.00%	54.25%
SST-2 macro F1	65.32%	40.74%
GoEmotions accuracy (12 mapped classes)	39.00%	35.67%
GoEmotions macro F1	27.34%	18.75%

For the SST-2 split, the reported TF-IDF model scores higher than the AST baseline in accuracy and macro F1. The same pattern appears in the GoEmotions accuracy and macro-F1 results. The AST baseline has the higher macro-precision value on GoEmotions (38.52% compared with 32.53%). This comparison shows the different aims of the two approaches: the trained model uses patterns learned from data, while the AST system makes its compositional rules visible. The reported values still need to be reproduced from the original experiment files.

TABLE VII. HELD-OUT DATASET BENCHMARKS (REPORTED VALUES)

Dataset / metric	TF-IDF ML	Compiler AST	Difference
SST-2 accuracy	67.00%	54.25%	ML +12.75 pp
SST-2 macro F1	65.32%	40.74%	ML +24.58 pp
GoEmotions accuracy	39.00%	35.67%	ML +3.33 pp
GoEmotions macro F1	27.34%	18.75%	ML +8.59 pp

B. Latency and Complexity

The project report records a mean end-to-end latency of 1.46 ms, a standard deviation of 120 µs, and a 99th-percentile latency of 2.16 ms across 1,000 iterations. It gives O(N) time complexity for tokenization, parsing, and traversal, where N is the number of tokens. These figures depend on the test machine and implementation. A benchmark script and details of the test environment are needed to reproduce them.

TABLE VIII. COMPUTATIONAL LATENCY AND COMPLEXITY (REPORTED)

Pipeline stage	Mean latency	99th percentile	Complexity
Lexical analysis	320 µs	480 µs	O(N)
Syntax / AST	450 µs	680 µs	O(N)
Semantic propagation	280 µs	390 µs	O(|AST|)
Aspect, emotion, audit	410 µs	610 µs	O(N)
End-to-end	1.46 ms	2.16 ms	O(N)

VI. CONCLUSION

This project uses compiler design stages for sentiment and emotion analysis. It separates tokenization, AST construction, semantic evaluation, and explanation generation. The AST records modifier scope and lets the system show how a sentiment score is formed. The project also includes aspect-level mixed sentiment, separate emotion labels, batch input, and optional local ML baselines. The reported verification results are higher for the AST system on the team's handcrafted cases, while the trained model scores better on the reported SST-2 and GoEmotions benchmarks. The next steps include testing with reproducible files, expanding the lexicon, and adding support for Universal Dependencies and other languages.

REFERENCES

[1] P. J. Stone, D. C. Dunphy, M. S. Smith, and D. M. Ogilvie, The General Inquirer: A Computer Approach to Content Analysis. Cambridge, MA, USA: MIT Press, 1966.

[2] M. Hu and B. Liu, “Mining and summarizing customer reviews,” in Proc. 10th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2004, pp. 168–177.

[3] M. Taboada, J. Brooke, K. Tofiloski, K. Voll, and M. Stede, “Lexicon-based methods for sentiment analysis,” Computational Linguistics, vol. 37, no. 2, pp. 267–307, 2011.

[4] C. J. Hutto and E. Gilbert, “VADER: A parsimonious rule-based model for sentiment analysis of social media text,” in Proc. 8th Int. AAAI Conf. Weblogs and Social Media, 2014, pp. 216–225.

[5] S. Baccianella, A. Esuli, and F. Sebastiani, “SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining,” in Proc. 7th Int. Conf. Language Resources and Evaluation, 2010, pp. 2200–2204.

[6] S. M. Mohammad and P. D. Turney, “Crowdsourcing a word-emotion association lexicon,” Computational Intelligence, vol. 29, no. 3, pp. 436–465, 2013.

[7] T. Wilson, J. Wiebe, and P. Hoffmann, “Recognizing contextual polarity in phrase-level sentiment analysis,” in Proc. EMNLP, 2005, pp. 347–354.

[8] X. Ding, B. Liu, and P. S. Yu, “A holistic lexicon-based approach to opinion mining,” in Proc. WSDM, 2008, pp. 231–240.

[9] B. Pang, L. Lee, and S. Vaithyanathan, “Thumbs up? Sentiment classification using machine learning techniques,” in Proc. EMNLP, 2002, pp. 79–86.

[10] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in Proc. EMNLP, 2013, pp. 1631–1642.

[11] K. S. Tai, R. Socher, and C. D. Manning, “Improved semantic representations from tree-structured long short-term memory networks,” in Proc. ACL, 2015, pp. 1556–1566.

[12] X. Zhu, P. Sobhani, and X. Guo, “Long short-term memory over recursive structures for sentiment analysis,” in Proc. IJCAI, 2015, pp. 1604–1610.

[13] P. Ekman, “An argument for basic emotions,” Cognition & Emotion, vol. 6, no. 3–4, pp. 169–200, 1992.

[14] R. Plutchik, “A general psychoevolutionary theory of emotion,” in Theories of Emotion. New York, NY, USA: Academic Press, 1980, pp. 3–33.

[15] D. Demszky et al., “GoEmotions: A dataset of fine-grained emotions,” in Proc. ACL, 2020, pp. 4040–4054.

[16] E. Cambria, S. Poria, R. Bajpai, and B. Schuller, “SenticNet 4: A two-level affective common sense ontology for sentiment analysis,” Knowledge-Based Systems, vol. 108, pp. 1–12, 2016.

[17] C. Strapparava and A. Valitutti, “WordNet-Affect: An affective extension of WordNet,” in Proc. LREC, 2004, pp. 1083–1086.

[18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[19] D. E. Knuth, “Semantics of context-free languages,” Mathematical Systems Theory, vol. 2, no. 2, pp. 127–145, 1968.

[20] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[21] P. Moilanen and S. Pulman, “Sentiment composition,” in Proc. Recent Advances in Natural Language Processing, 2007, pp. 378–382.

[22] L. Polanyi and A. Zaenen, “Contextual valence shifters,” in Computing Attitude and Affect in Text: Theory and Applications. Dordrecht, Netherlands: Springer, 2006, pp. 1–10.

[23] L. Jia, C. Yu, and W. Meng, “The effect of negation on sentiment analysis and retrieval effectiveness,” in Proc. ACM CIKM, 2009, pp. 1827–1830.

[24] Y. Choi and C. Cardie, “Learning with compositional semantics as structural inference for subsentential sentiment analysis,” in Proc. EMNLP, 2008, pp. 793–801.

[25] T. Nakagawa, K. Inui, and S. Kurohashi, “Dependency tree-based sentiment classification using CRFs with hidden variables,” in Proc. COLING, 2010, pp. 786–794.

[26] M. Pontiki et al., “SemEval-2014 Task 4: Aspect based sentiment analysis,” in Proc. 8th Int. Workshop on Semantic Evaluation, 2014, pp. 27–35.

[27] M. Pontiki et al., “SemEval-2016 Task 5: Aspect based sentiment analysis,” in Proc. 10th Int. Workshop on Semantic Evaluation, 2016, pp. 19–30.

[28] B. Liu, Sentiment Analysis: Mining Opinions, Sentiments, and Emotions. Cambridge, UK: Cambridge University Press, 2015.

[29] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[30] R. Guidotti et al., “A survey of methods for explaining black box models,” ACM Computing Surveys, vol. 51, no. 5, Art. no. 93, 2018.

[31]Team 9 original project PDF. Compiler-Based Sentiment Analysis Using Lexical, Syntax and Semantic Analysis. Use the exact title and authors printed on your PDF's title page.